Algoritmo

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a mathlab y permite el trabajo con matrices.

```
a=38
a=40
b=80
c=b-a
c
```

```
## [1] 40
```

```
y=40
alpha<-30
```

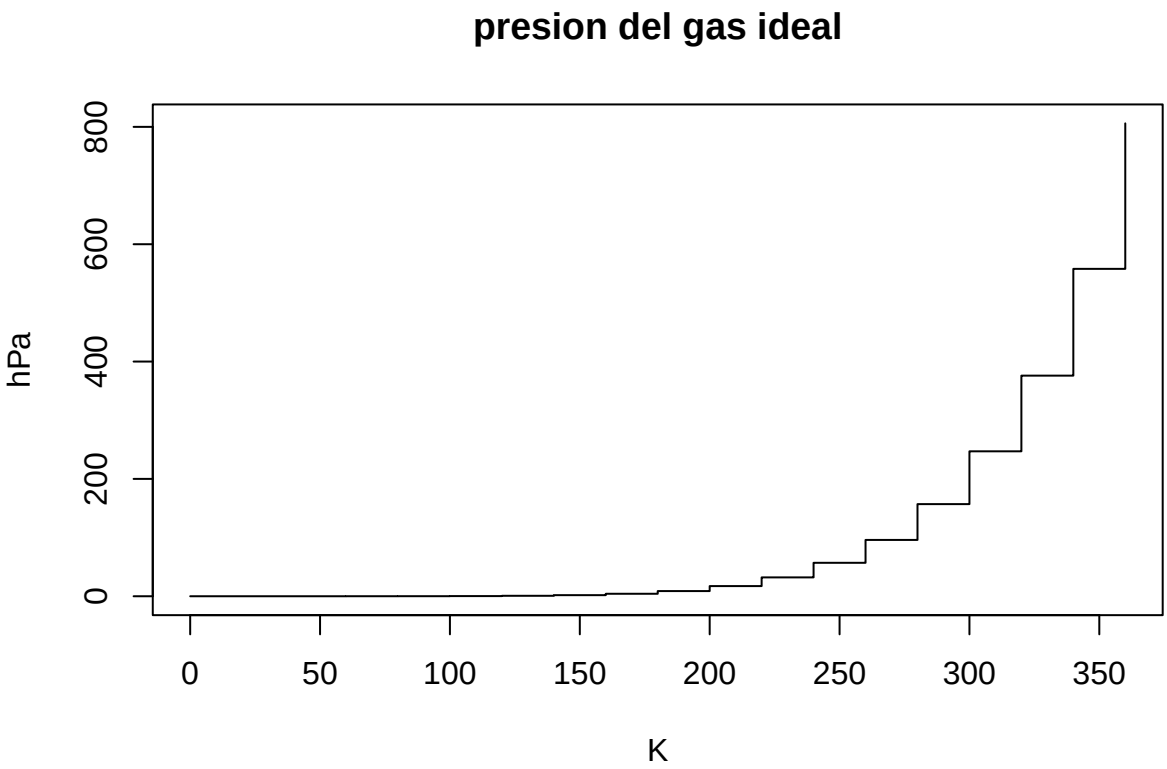
##uso de datasets o base de datos internos de R usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $ depues del comando de informacion especifico y elegimos la columa de datos deseada

```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

Including Plots

You can also embed plots, for example:



##comando plot El comando plot es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar un comando podemos escribir ##plot## en la consola, lo cual abrira una ventana que explicara en detalle el comando.

##Funciones estadisticas

```
z1<-rnorm(350,22,5)
z1
```

```
##   [1] 17.754979 27.041217 17.074430 24.900313 12.891667 19.522212 22.504271
##   [8] 22.490449 20.042311 25.309946 20.520144 24.604571 20.274620 21.657621
##  [15] 22.631626 33.254096 21.658366 24.621414 29.503066 12.172990 25.891689
##  [22] 17.788485 14.228617 20.961472 21.047895 21.837039 20.832671 22.661050
##  [29] 34.365250 22.236124 29.644630 20.113886 14.909934 26.219940 28.977563
##  [36] 26.475969 16.116281 26.572979 19.468032 25.743141 34.361027 12.273838
##  [43] 16.313971 25.595509 17.589000 16.905794 29.981063 23.319548 29.819589
##  [50]  9.505294 13.045615 28.547699 23.623362 22.265779 23.343153 18.660258
##  [57] 18.078772 18.223458 19.820514 23.733911 16.629150 24.755004 22.927372
##  [64] 32.295854 21.734202 26.339242 17.909080  7.494946 23.412112 21.853280
##  [71] 23.575241 20.971676 30.413163 14.150202 18.872650 21.938432 24.350289
##  [78] 24.006602 20.259650 22.403000 19.956412 15.095580 17.838134 25.484331
##  [85] 28.585074 24.481876 32.268885 22.563298 16.817843 19.554628 19.962296
##  [92] 14.897788 17.000306 22.642580 20.418219 13.543372 21.104392 26.685794
##  [99] 25.238241 27.404645 21.536355 19.571105 21.151155 20.513631 17.272681
## [106] 26.665331 23.239235 25.598536 25.792985 16.917550 25.717226 31.023400
## [113] 24.374134 27.153396 22.017757 27.901037 16.697909 29.358556 25.595108
## [120] 24.725418 14.255012 20.728652  9.919358 22.345506 21.413163 22.665968
## [127] 20.643930 23.906772 20.695463 21.537647 14.852258 25.132288 20.223789
## [134] 26.283504 18.084463 20.367979 23.176444 21.704947 16.841280 25.126101
## [141] 20.364144 23.092265 19.950599 24.002681 19.707216 24.723615 21.559114
```

```
## [148] 26.897428 20.878036 22.966881 18.726579 19.037607 30.159563 26.471700
## [155] 28.017799 22.585386 23.709460 25.327389 14.004593 24.039474 26.801468
## [162] 12.177261 25.345287 19.074078 17.714135 15.304761 18.194023 19.714745
## [169] 21.823708 19.083888 16.326409 23.696136 13.275714 17.451447 23.953551
## [176] 38.870321 28.946945 17.835513 19.266018 19.614397 17.769782 16.022013
## [183] 17.849966 23.619039 24.829453 28.455815 13.897766 26.371267 27.226801
## [190] 19.259888 22.484796 25.026257 20.841383 29.515532 30.223801 23.393054
## [197] 15.425035 19.838027 16.017693 13.540109 15.866041 25.330379 28.575085
## [204] 19.960986 23.602799 31.464043 18.462906 19.992956 25.310483 18.598179
## [211] 21.312963 24.427961 30.536831 19.349243 28.045564 19.117041 18.187671
## [218] 19.950736 22.038094 17.687905 26.749936 23.961261 25.420203 20.938446
## [225] 14.664441 17.925001 17.095600 26.129996 18.536952 28.818655 21.788696
## [232] 24.550976 28.890161 18.681893 22.665251 29.244177 22.868495 26.721084
## [239] 21.698729 24.376090 21.550109 24.221775 18.073051 26.743841 19.806649
## [246] 27.009440 27.635448 23.919218 30.470890 22.617410 20.564417 19.567541
## [253] 18.455713 26.289311 26.372086 24.900289 23.941639 17.344252 16.064234
## [260] 23.567517 17.932906 24.276321 31.543887 27.441025 24.799942 20.667219
## [267] 24.151801 26.825500 33.189236 25.467348 18.302525 21.114747 19.175969
## [274] 21.022336 16.925031 21.817767 20.863017 16.606858 25.329470 21.712709
## [281] 19.002657 23.350440 16.809295 22.180915 25.187437 15.790952 21.952545
## [288] 21.588849 20.584658 19.458925 12.290064 16.410488 28.256060 20.956859
## [295] 22.346621 21.012695 18.712641 22.972291 27.154831 15.734790 20.312702
## [302] 24.409942 26.187088 23.715878 27.897168 25.558802 24.050499 20.376834
## [309] 23.788623 18.315452 16.266317 16.686915 25.372790 29.828543 33.144151
## [316]  9.065662 15.402617 21.817444 27.141014 21.176464 25.253113 33.281159
## [323] 18.595244 24.114339 28.867503 22.871974 18.521017 22.879761 18.001116
## [330] 20.812526 18.320123 29.771902 32.154780 20.237251 20.826667 22.093332
## [337] 18.715112 14.167041 20.384698 25.817236 26.641123 23.295216 25.186130
## [344] 26.386296 23.765098 28.472199 21.613762 23.636992 21.983576 21.345424
```

```r
w1<-length(z1)
w1
```

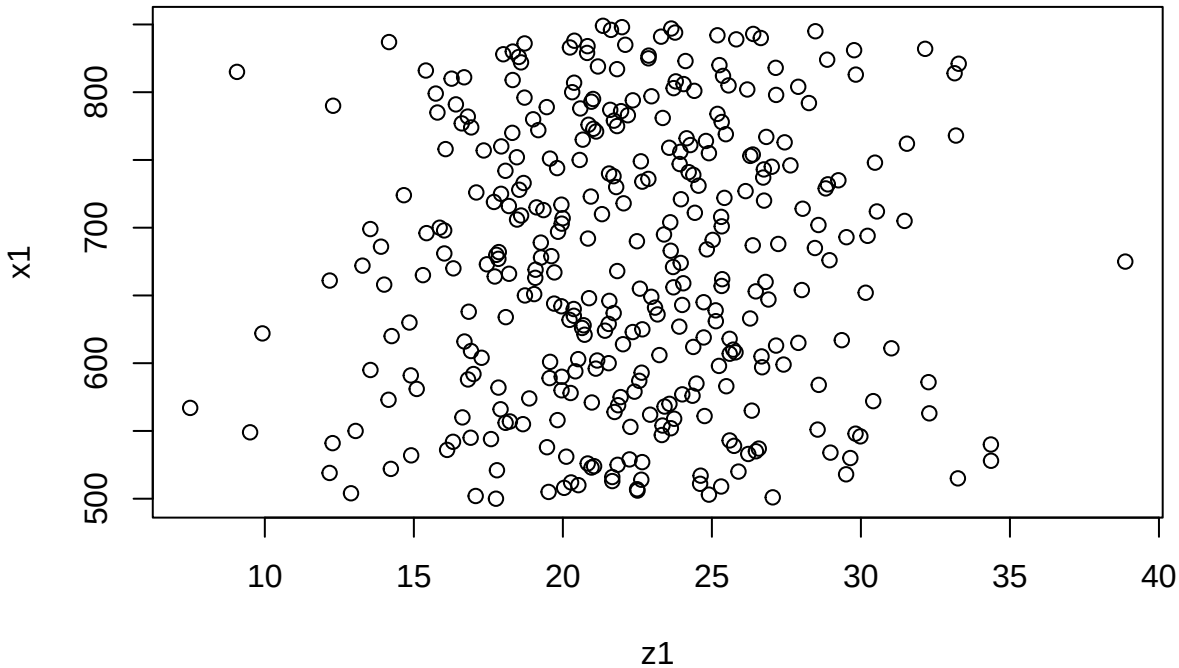
```
## [1] 350
```

```r
x1<-(500:849)
x1
```

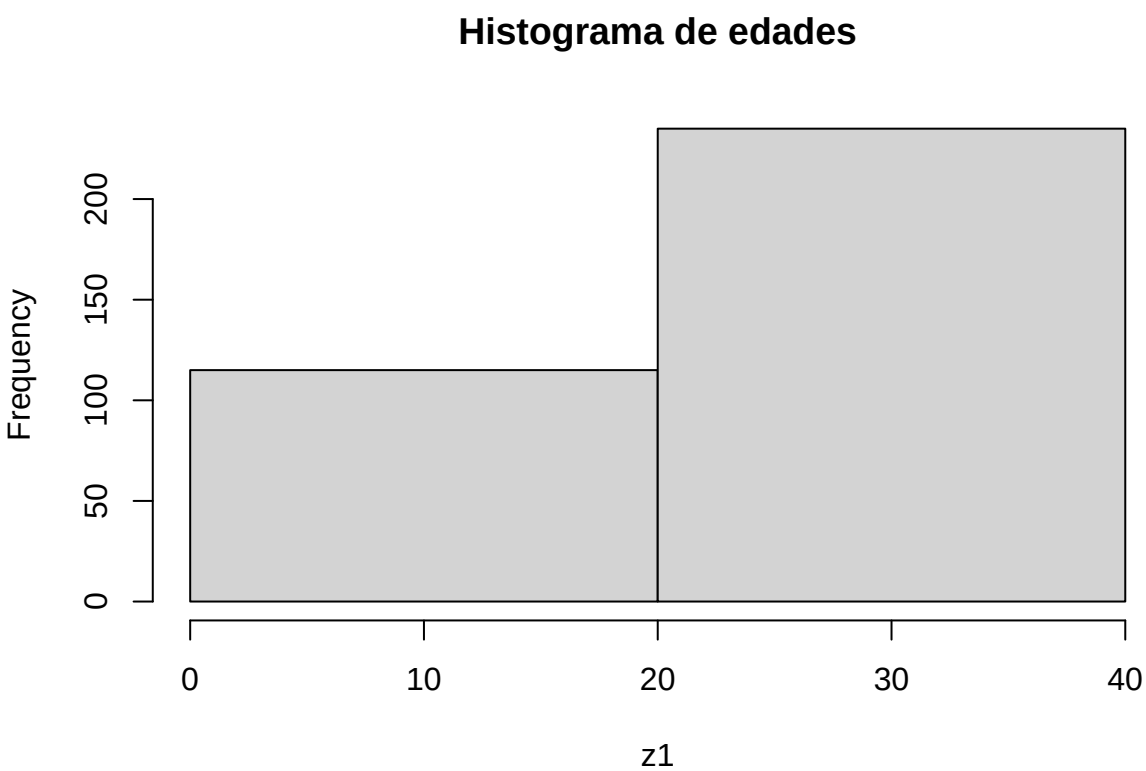
```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
```

```
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1,x1)
```




```r
hist(z1,main="Histograma de edades",breaks=2)
```



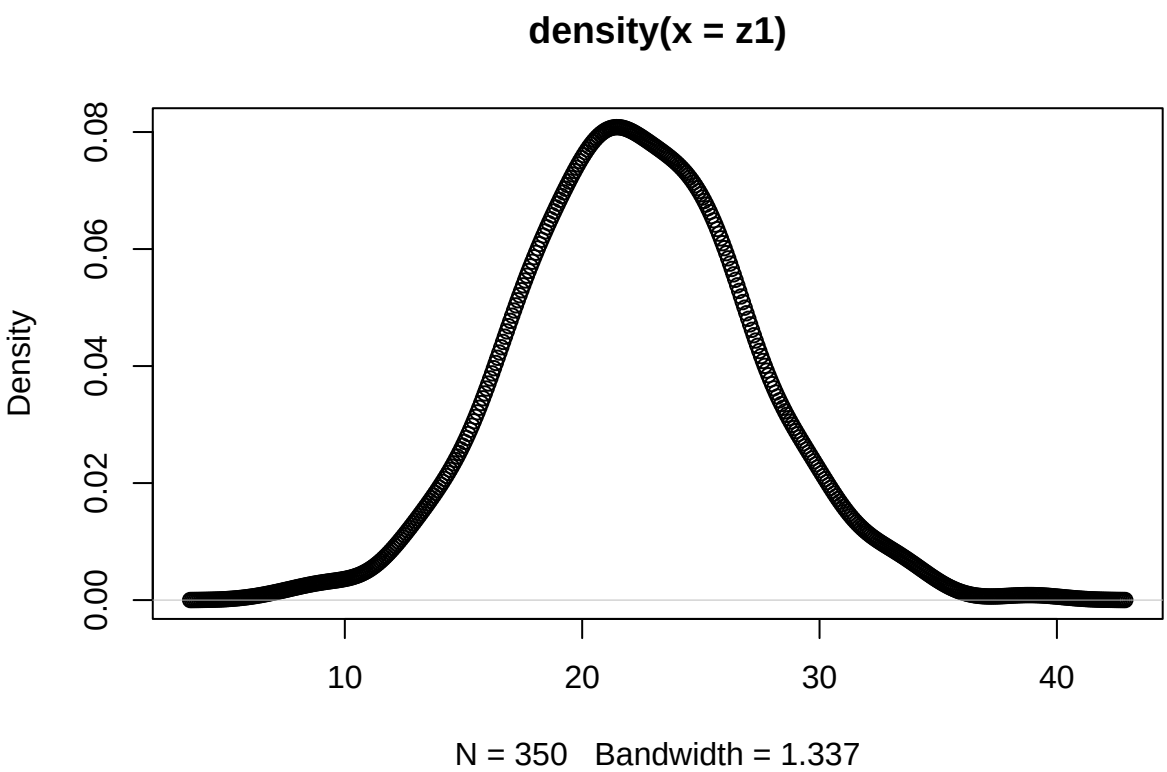

```r
density(z1,type='d')
```

```
## Warning: In density.default(z1, type = "d") :
```

```
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "d")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.337
##
##        x                y
##  Min.   : 3.484   Min.   :9.572e-06
##  1st Qu.:13.333   1st Qu.:1.036e-03
##  Median :23.183   Median :1.090e-02
##  Mean   :23.183   Mean   :2.533e-02
##  3rd Qu.:33.032   3rd Qu.:4.871e-02
##  Max.   :42.881   Max.   :8.084e-02
```

```
plot(density(z1),type='b')
```




```
DNI=477
```

##Ejercicio 1 Consigna: Crear un vector que tenga todos los numeros enteros desde el 1 hasta DNI

```
vector_DNI=(1:DNI)
vector_DNI
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
```

```
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477
```

##Ejercicio 2 Calcular la suma de todos los valores del vector

```
Total=0
Valor_final=length(vector_DNI)
for (i in 1:Valor_final) {
  Total=Total+i}
  Total
```

```
## [1] 114003
```

##Ejercicio 3 Repetir el ejercicio anterior, pero en python

```
suma=0
DNI=477
for i in range(1, DNI):
  suma +=i
print(suma)
```

```
## 113526
```

##Ejercicio 4 ¿Cuanto tarda en correr el codigo del ejercicio 2?

```
Inicio=Sys.time()
Total=0
Valor_final=length(vector_DNI)
for (i in 1:Valor_final) {
  Total=Total+i}
  Total
```

```
## [1] 114003
```

```
Fin=Sys.time()
Fin-Inicio
```

```
## Time difference of 0.004517078 secs
```

##Ejercio 5 Calcular el ejercicio anterior usando la cabeza de gauss

```r
n=length(vector_DNI)
suma=n*(n+1)/2
print(suma)
```

```
## [1] 114003
```

##CONSIGNA: Comparar la performance con systime

```r
A <- 0
InicioA=Sys.time()
for (i in 1:50000) { A[i] <- (i*2)}
head (A)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```r
FinA=Sys.time()
FinA-InicioA
```

```
## Time difference of 0.0229547 secs
```

```r
InicioB=Sys.time()
B=seq(1, 100000, by = 2)
head(B)
```

```
## [1]  1  3  5  7  9 11
```

```r
tail(B)
```

```
## [1] 99989 99991 99993 99995 99997 99999
```

```r
FinB=Sys.time()
FinB-InicioB
```

```
## Time difference of 0.00252223 secs
```

##CONSIGNA: Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000 ?

```r
a0=0
a1=1
it=2
  while (a1<1000001){
    an=a0+a1
    a0=a1
    a1=an
    it=it+1}
it
```

```
## [1] 32
```

##CONSIGNA: Compara la performance de ordenación del método burbuja vs el método sort de R. REVISAR COMO SE USA!!!!!! Arreglar

```r
library(microbenchmark)
# Tomo una muestra de 10 números ente 1 y 100
x<-sample(1:100,10)
# Creo una funci?n para ordenar
burbuja <- function(x){
n<-length(x)
```

```r
for(j in 1:(n-1)){
for(i in 1:(n-j)){
if(x[i]>x[i+1]){
temp<-x[i]
x[i]<-x[i+1]
x[i+1]<-temp
}
}
}
return(x)
}
res<-burbuja(x)
Bb<- microbenchmark(NULL, burbuja(x), times=1000L)
#Muestra obtenida
x
```

```
##  [1] 77 45 24 66  2  7 69 22 80 67
```

```r
#Muestra Ordenada
res
```

```
##  [1]  2  7 22 24 45 66 67 69 77 80
```

```r
#Ordanación con el coamando SORT de R-Cran
sort(x)
```

```
##  [1]  2  7 22 24 45 66 67 69 77 80
```

```r
Bs<- microbenchmark(NULL, sort(x), times=1000L)
#Benchmarks
Bb
```

```
## Unit: nanoseconds
##        expr  min   lq     mean median   uq   max neval
##        NULL    0    0    8.477     10   10   100  1000
##  burbuja(x) 6029 6240 6729.560   6380 6675 38210  1000
```

```r
Bs
```

```
## Unit: nanoseconds
##     expr   min    lq      mean median    uq    max neval
##     NULL     0     0     8.384     10    10    140  1000
##  sort(x) 21971 23415 25670.334  24160 25005 124890  1000
```